

BÀI 2 CÁC THÀNH PHẦN CƠ SỞ CỦA JAVA

(Số tiết: 5 tiết)

Mục tiêu:

- Hiểu được cấu trúc chung của chương trình ứng dụng Java độc lập
- Hiểu được các phần tử cơ bản trong Java như: định danh, từ khóa, chú thích, các phép toán, các kiểu dữ liệu nguyên thủy, mảng và các câu lệnh điều khiển của Java.
- Áp dụng được các kiến thức đã cung cấp để lập trình giải các bài toán đơn giản.

Nội dung chính:

- Giới thiệu về Java
- Các dạng chương trình ứng dụng Java
- Cấu trúc chương trình ứng dụng độc lập
- Biên dịch, thông dịch và thực thi ứng dụng Java
- Các phần tử cơ bản của java

2.1 Giới thiệu về Java

❖ 2.1.1. Lịch sử hình thành và phát triển của Java

Java là ngôn ngữ lập trình hướng đối tượng (tựa C++) do Sun Microsystem đưa ra vào giữa thập niên 90. Chương trình viết bằng ngôn ngữ lập trình java có thể chạy trên bất kỳ hệ thống nào có cài máy ảo java

Năm 1990: James Gosling và các công sự của Công ty Sun Microsystem tham gia dự án Green Team - xây dựng công nghệ mới cho ngành điện tử tiêu dung (cho các thiết bị điện dân dụng). Để giải quyết vấn đề này nhóm nghiên cứu phát triển đã xây dựng một ngôn ngữ lập trình mới đặt tên là Oak tương tự như C++ nhưng loại bỏ một số tính năng nguy hiểm của C++ và có khả năng chạy trên nhiều nền phần cứng khác nhau.

Năm 1993 world wide web bắt đầu phát triển. Năm 1994, Sun đưa ra trình duyệt web viết bằng ngôn ngữ oak là webrunner, sau đổi tên thành hostJava.

Sau đó, Sun đổi tên oak thành Java và được giới thiệu năm 1995 tại Sunworld 1995.

Java là tên gọi của một hòn đảo ở Indonexia, đây là nơi nhóm nghiên cứu phát triển đã chọn để đặt tên cho ngôn ngữ lập trình Java trong một chuyến đi tham quan và làm việc trên hòn đảo này. Hòn đảo Java này là nơi rất nổi tiếng với nhiều khu vườn trồng cafe, đó chính là lý do chúng ta thường thấy biểu tượng ly cafe trong nhiều sản phẩm phần mềm, công cụ lập trình Java của Sun cũng như một số hãng phần mềm khác đưa ra.

❖ 2.1.2. Các đặc trưng của chương trình Java

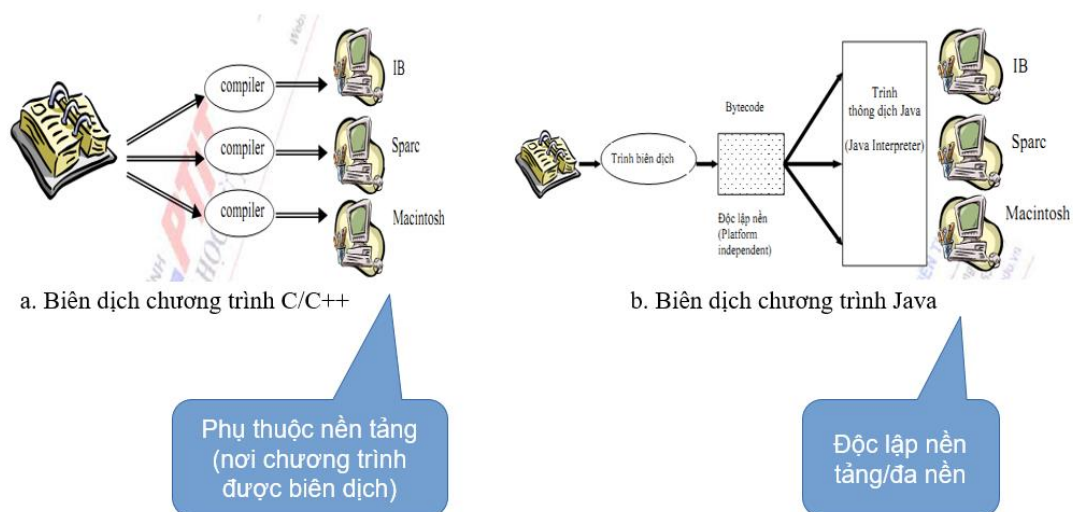
Đơn giản: phát triển trên một ngôn ngữ dễ học và quen thuộc với đa số người lập trình là C và Java loại bỏ các phức tạp của C và C++ như: con trỏ, đa kế thừa, định nghĩa chồng toán tử, không sử dụng lệnh “goto” cũng như file header (.h), loại bỏ cấu trúc “struct” và “union”.

Hướng đối tượng Java là ngôn ngữ lập trình hoàn toàn hướng đối tượng:

- Mọi thực thể trong hệ thống đều được coi là một đối tượng, tức là một thể hiện cụ thể của một lớp xác định.
- Tất cả các chương trình đều phải nằm trong một class nhất định.
- Không thể dùng Java để viết một chức năng mà không thuộc vào bất kì một lớp nào. Tức là Java không cho phép định nghĩa dữ liệu và hàm tự do trong chương trình.

Độc lập phần cứng và hệ điều hành

Đối với các ngôn ngữ lập trình truyền thống như C/C++, phương pháp biên dịch được thực hiện như sau:



Hình 2.1 Chương trình C, và java – độc lập với nền tảng

Với một nền phần cứng khác nhau, có một trình biên dịch khác nhau để biên dịch mã nguồn chương trình cho phù hợp với nền phần cứng ấy. Do vậy, khi chạy trên một nền phần cứng khác, bắt buộc phải biên dịch lại mã nguồn.

Đối các chương trình viết bằng Java, chỉ cần cài máy ảo java là có thể chạy được.

Máy ảo Java (Java Virtual Machine).

- Là một phần mềm dựa trên cơ sở máy tính ảo
- Là tập hợp các lệnh logic để xác định hoạt động của máy tính
- Được xem như là một hệ điều hành thu nhỏ
- Nó thiết lập lớp trừu tượng cho:
 - Phần cứng bên dưới
 - Hệ điều hành

- Mã đã biên dịch

Mạnh mẽ Java là ngôn ngữ yêu cầu chặt chẽ về kiểu dữ liệu:

- Kiểu dữ liệu phải được khai báo tường minh.
- Java không sử dụng con trỏ và các phép toán con trỏ.
- Java kiểm tra việc truy nhập đến mảng, chuỗi khi thực thi để đảm bảo rằng các truy nhập đó không ra ngoài giới hạn kích thước mảng.
- Quá trình cấp phát, giải phóng bộ nhớ cho biến được thực hiện tự động, nhờ dịch vụ thu nhặt những đối tượng không còn sử dụng nữa (garbage collection).
- Cơ chế bắt lỗi của Java giúp đơn giản hóa quá trình xử lý lỗi và hồi phục sau lỗi.

Bảo mật Java cung cấp một môi trường quản lý thực thi chương trình với nhiều mức để kiểm soát tính an toàn:

- Ở mức thứ nhất, dữ liệu và các phương thức được đóng gói bên trong lớp. Chúng chỉ được truy xuất thông qua các giao diện mà lớp cung cấp.
- Ở mức thứ hai, trình biên dịch kiểm soát để đảm bảo mã là an toàn, và tuân theo các nguyên tắc của Java.
- Mức thứ ba được đảm bảo bởi trình thông dịch. Chúng kiểm tra xem bytecode có đảm bảo các qui tắc an toàn trước khi thực thi.
- Mức thứ tư kiểm soát việc nạp các lớp vào bộ nhớ để giám sát việc vi phạm giới hạn truy xuất trước khi nạp vào hệ thống.

Phân tán Java được thiết kế để hỗ trợ các ứng dụng chạy trên mạng bằng các lớp Mạng (java.net). Hơn nữa, Java hỗ trợ nhiều nền chạy khác nhau.

Đa luồng Java cung cấp giải pháp đa luồng (Multithreading) để thực thi các công việc cùng đồng thời và đồng bộ giữa các luồng.

Linh động Java được thiết kế như một ngôn ngữ động để đáp ứng cho những môi trường mở. Các chương trình Java chứa rất nhiều thông tin thực thi nhằm kiểm soát và truy nhập đối tượng lúc chạy.

2.2 Các dạng chương trình ứng dụng Java

Ứng dụng độc lập

Các ứng dụng Java độc lập là các chương trình được viết bằng Java để thực hiện các tác vụ nhất định. Các ứng dụng này chạy trực tiếp bởi Java. Một ứng dụng độc lập được cài đặt trên mỗi máy tính trước khi chạy. Nó có thể có giao diện dòng lệnh (CLI) hoặc Giao diện người dùng (GUI). Ví dụ như trình soạn thảo văn bản, trình xử lý văn bản và ứng dụng trò chơi.

Ví dụ 1: gõ đoạn mã sau:

```
package Hello;
import java.io.*;
public class HelloWorldApp{
    public static void main(String[] args){
        System.out.println("HelloWorld");
    }
}
```

```
}  
}
```

Lưu lại với tên HelloWorldApp.java

Để biên dịch mã nguồn, ta sử dụng trình biên dịch javac.

- Mở cửa sổ Command Prompt.
- Chuyển đến thư mục chứa tập tin nguồn vừa tạo ra.
- Thực hiện câu lệnh: **javac HelloWorldApp.java**

Trình dịch javac tạo ra file **HelloWordApp.class** chứa các mã “bytecodes”. Để chương trình thực thi được ta cần dùng trình thông dịch:

java HelloWorldApp

ta sẽ thấy dòng chữ HelloWorld trên màn hình Console.

Ứng dụng nhúng applet

Applet là một chương trình Java có thể chạy trong các trình duyệt web có hỗ trợ Java. Tất cả các applet đều là các lớp con của lớp Applet. Để tạo applet, ta cần import hai gói sau:

```
import java.applet.*;
```

```
import java.awt.*;
```

Cấu trúc của một Applet:

```
public class <Tên lớp applet> extends Applet{  
    ... // Các thuộc tính  
    public void init(){...}  
    public void start(){...}  
    public void stop(){...}  
    public void destroy(){...}  
    ... // Các phương thức khác  
}
```

Các phương thức cơ bản của một applet:

- init(): Khởi tạo các tham số, nếu có, của applet.
- start(): Applet bắt đầu hoạt động
- stop(): Chấm dứt hoạt động của applet.
- destroy(): Thực hiện các thao tác dọn dẹp trước khi thoát khỏi applet.

Lưu ý:

- Không phải tất cả các applet đều phải cài đặt đầy đủ 4 phương thức cơ bản trên. Applet còn có thể cài đặt một số phương thức tùy chọn (không bắt buộc) sau:
- paint(Graphics): Phương thức vẽ các đối tượng giao diện bên trong applet. Các thao tác vẽ này được thực hiện bởi đối tượng đồ họa Graphics (là tham số đầu vào).

- `repaint()`: Dùng để vẽ lại các đối tượng trong applet. Phương thức này sẽ tự động gọi phương thức `update()`.
- `update(Graphics)`: Phương thức này được gọi sau khi thực hiện phương thức `paint` nhằm tăng hiệu quả vẽ. Phương này sẽ tự động gọi phương thức `paint()`.

Ví dụ 2: cài đặt một applet đơn giản vẽ ra chuỗi “hello word!”

```
import java.awt.*;
import java.applet.*;
public class SimpleApplet extends Applet{
    public void paint(Graphics g){
        g.drawString( “ Hello world!”,50,50);
    }
}
```

Applet không thể chạy như một ứng dụng Java độc lập (nó không có hàm main), mà nó chỉ chạy được khi được nhúng trong một trang HTML (đuôi .htm, .html) và chạy bằng một trình duyệt web thông thường.

Các bước xây dựng và sử dụng một applet bao gồm:

- Biên dịch mã nguồn thành lớp .class
- Nhúng mã .class của applet vào trang html.

Để nhúng một applet vào một trang html, ta dùng thẻ (tag) `<Applet>`.

Trong trang `myHtml.htm` có chứa nội dung như sau:

```
<HTML>
<HEAD>
<TITLE> A simple applet </TITLE>
</HEAD>
<BODY> This is the output of applet:
    <APPLET CODE=“SimpleApplet.class” WIDTH=200 HEIGHT=20>
    </APPLET>
</BODY>
</HTML>
```

Mở trang `myHtml` trên các trình duyệt thông thường hoặc dùng lệnh `appletviewer` để chạy

2.3 Cấu trúc chương trình ứng dụng độc lập

Cấu trúc chung:

```

package <package_name>;
import <other_package>;
public class ClassName {
    <Variables (also known as fields)>;
    <constructor method(s)>;
    <other methods>;
}

```

Trong đó:

- **package:** Một package (gói) mô tả không gian tên có chứa các lớp của Java, sử dụng ký tự thường và dấu chấm để định nghĩa tên, chúng ta có thể xem package như là một thư mục, còn class chính là các file trực thuộc thư mục.
- **import:** Từ khóa được sử dụng trong Java nhằm để xác định các class hoặc các package được sử dụng trong lớp này.
- **class:** Từ khóa nhằm để định nghĩa lớp của Java. Nó đứng trước khai báo tên lớp của Java. Ngoài ra còn có từ khóa public, từ khóa này xác định phạm vi truy cập của lớp. Đặc tính này chính là tính đóng gói trong lập trình hướng đối tượng. (chúng ta sẽ tìm hiểu phần này ở các bài sau)
- **variables:** Biến hay còn gọi là trường, cũng có một số tài liệu gọi là thuộc tính trực thuộc lớp. Nó chứa thông tin cụ thể liên quan tới các đối tượng là thể hiện của lớp.
- **methods:** Phương thức hay còn gọi là hàm chứa các hành động thực thi của đối tượng. Đương nhiên nội dung của phương thức chính là các đoạn mã thực thi của chính phương thức này.
- **constructors :** Phương thức khởi tạo (Hay hàm khởi tạo) của đối tượng. Hình dạng của đối tượng được thể hiện ra sao sẽ phụ thuộc vào phương thức này.

Ví dụ 1:

```

package Hello;
import java.io.*;
public class HelloWorldApp{
    public static void main(String[] args){
        System.out.println("HelloWorld");
    }
}

```

2.4 Biên dịch, thông dịch và thực thi ứng dụng Java

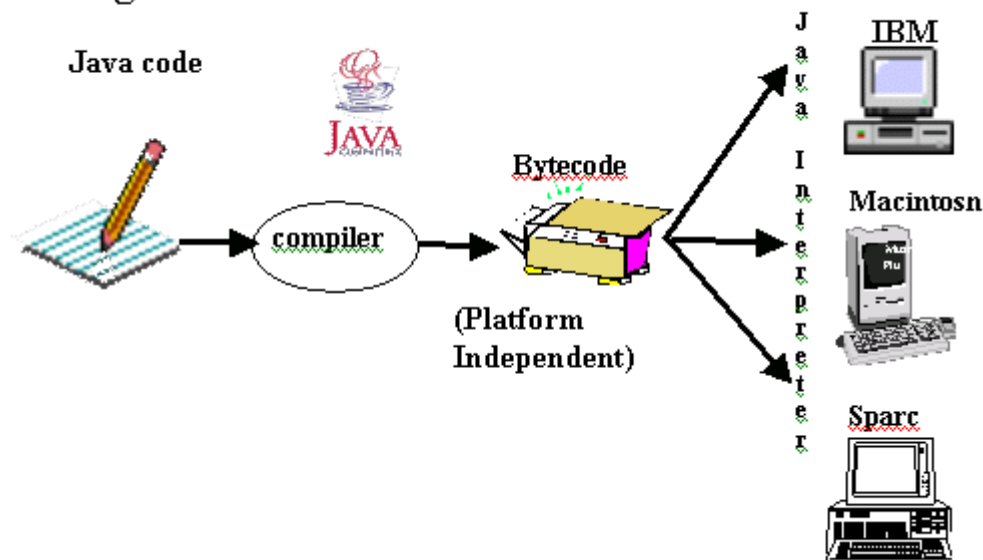
Ngôn ngữ lập trình thường được chia ra làm 2 loại (tùy theo các hiện thực hóa ngôn ngữ đó) là ngôn ngữ thông dịch (Interpreted Language) và ngôn ngữ biên dịch (Compiled Language).

- Thông dịch (Interpreter) : Nó dịch từng lệnh rồi chạy từng lệnh, lần sau muốn chạy lại thì phải dịch lại.
- Biên dịch (Compiler): Code sau khi được biên dịch sẽ tạo ra 1 file thường là .exe, và file .exe này có thể đem sử dụng lại không cần biên dịch nữa.

Ngôn ngữ lập trình Java thuộc loại ngôn ngữ thông dịch. Chính xác hơn, Java là loại ngôn ngữ vừa biên dịch (Interpreted Language) vừa thông dịch. Khi viết mã, hệ thống tạo ra một tệp .java. Khi biên dịch mã nguồn của chương trình sẽ được biên dịch ra mã byte code. Máy ảo Java (Java Virtual Machine) sẽ thông dịch mã byte code này thành machine code (hay native code) khi nhận được yêu cầu chạy chương trình. Quá trình dịch chương trình Java:

- Trình biên dịch chuyển mã nguồn thành tập các lệnh không phụ thuộc vào phần cứng cụ thể.
- Trình thông dịch trên mỗi máy chuyển tập lệnh này thành chương trình thực thi
- Máy ảo tạo ra một môi trường để thực thi các lệnh bằng cách:
 - o Nạp các file .class
 - o Quản lý bộ nhớ
 - o Dọn “rác”

Programs



Hình 2.2 Chạy và biên dịch chương trình java

2.5 Các phần tử cơ bản của java

2.5.1 Định danh, từ khóa, chú thích

Định danh (Tên gọi)

Trong Java định danh là một dãy các ký tự gồm các chữ cái, chữ số và một số các ký hiệu như: ký hiệu gạch dưới nối câu "_", các ký hiệu tiền tệ \$, O, Ê, Â, và không được bắt đầu bằng chữ số.

Java phân biệt chữ thường và chữ hoa. Độ dài (số ký tự) của định danh trong Java về lý thuyết là không bị giới hạn.

Quy ước (lời khuyên):

- Định danh được đặt tên cho các lớp, các đối tượng: chữ cái đầu của mỗi từ viết hoa.

- Định danh cho các biến, phương thức, tên phương thức thường bằng động từ (viết in thường, và các danh từ phía sau thì chữ cái đầu viết chữ in hoa) và , đối tượng: chữ cái đầu của mỗi từ trong định danh đều viết hoa trừ từ đầu tiên (“náo”).

Chú thích (Comment)

```
// Chú thích trên một dòng

/* Chú thích trên nhiều dòng

... */

/** Chú thích trong tư liệu (javadoc)

... */
```

Các từ khóa của java

Bảng 2.1 Các từ khóa của Java

abstract	boolean	break	byte
case	catch	char	class
const	continue	default	do
double	else	extends	final
finally	float	for	goto
if	implements	import	instanceof
int	interface	long	native
new	package	private	protected
public	return	short	static
super	switch	synchronized	this
throw	throws	transient	try
void	volatile	while	

Các kí tự định dạng xuất dữ liệu (Escape Sequences)

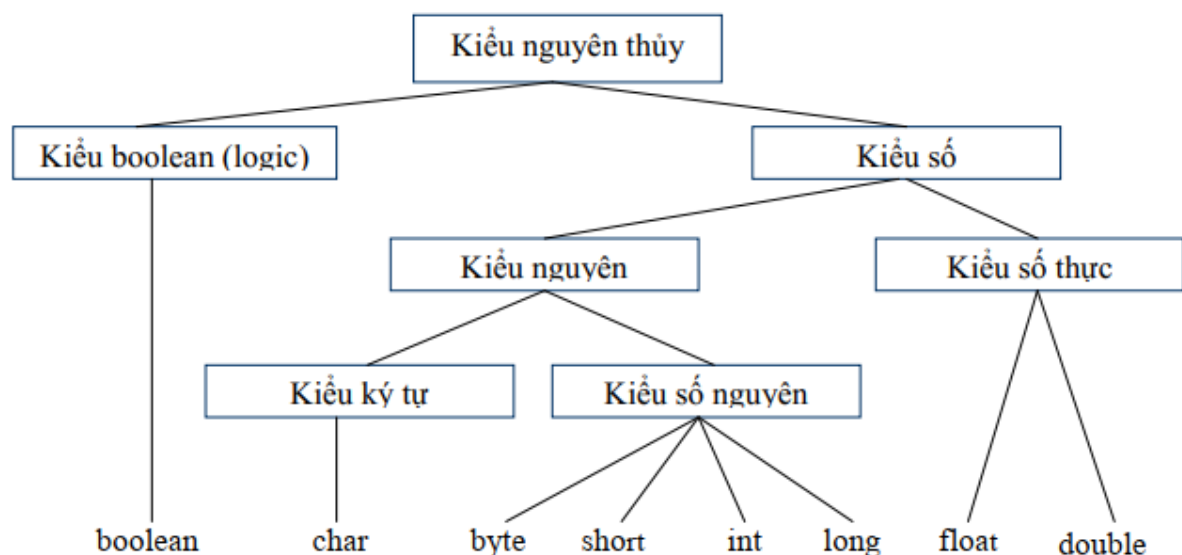
Bảng 2.2 Các kí tự định dạng xuất dữ liệu

Escape Sequence	Mô tả
\n	Xuống dòng mới
\r	Chuyển con trỏ đến đầu dòng hiện hành
\t	Chuyển con trỏ đến vị trí dừng Tab kế tiếp (ký tự Tab)
\\	In dấu \
\'	In dấu nháy đơn (')
\''	In dấu nháy kép (")

2.5.2 Các kiểu dữ liệu trong Java

Java có 2 dạng dữ liệu: Dữ liệu nguyên thủy và dữ liệu tham chiếu đối tượng.

Các kiểu dữ liệu nguyên thủy là các kiểu dữ liệu có sẵn và định nghĩa trước của java:



Hình 2.1 Các kiểu dữ liệu nguyên thủy trong Java

Mỗi kiểu nguyên thủy có một lớp bao bọc (Wrapper hay còn gọi lớp nguyên thủy) tương ứng để sử dụng các giá trị nguyên thủy như là các đối tượng. Ví dụ ứng với kiểu int có lớp Integer, ứng với char là Char vv.. các lớp này sẽ cung cấp một số phương thức hỗ trợ trên kiểu tương ứng.

2.5.3 Các phép toán cơ bản trong Java

Toán tử số học

Bảng 2.1 Toán tử số học

Toán tử	Ý nghĩa
+	Cộng
-	Trừ
*	Nhân
/	Chia
%	Chia dư
++	Tăng 1
--	Giảm 1

- Phép chia nguyên (2 toán hạng đều là kiểu nguyên) đòi hỏi số chia phải khác 0.
- Phép chia số thực (ít nhất một toán hạng kiểu số thực) cho phép chia cho 0 và kết quả phép chia cho 0.0 là INF (số lớn vô cùng) hoặc -INF (số âm vô cùng), hai hằng đặc biệt trong Java.
- Trong Java, phép chia lấy số dư % thực hiện được cả đối với số thực. (float m = 11.5 % 2.5; // Cho m là 1.5)
- Khi sử dụng các phép toán đơn nguyên đối với các đối số kiểu byte, short, hoặc char thì trước tiên toán hạng phải được tính rồi chuyển về kiểu int và kết quả là kiểu int.

Ví dụ 5:

short h = 30; // OK: 30 kiểu int chuyển về short (mặc định đối với hằng nguyên)
h = h + 4; // Lỗi vì h + 4 cho kết quả kiểu int không thể gán cho h kiểu short phải ép kiểu: h = (short) (h+4);

Toán tử quan hệ & logic

Bảng 2.2 Toán tử quan hệ & logic

Toán tử	Ý nghĩa
=	So sánh bằng
!=	So sánh khác
>	So sánh lớn hơn
<	So sánh nhỏ hơn
>=	So sánh lớn hơn hay bằng
<=	So sánh nhỏ hơn hay bằng
	OR (biểu thức logic)
&&	AND (biểu thức logic)
!	NOT (biểu thức logic)

Toán tử gán (assignment)

Bảng Error! No text of specified style in document..3 Toán tử gán

Toán tử	Ví dụ	Giải thích
---------	-------	------------

=	int c=3, d=5; c=4;	
+=	c+=7	c=c+7
-=	d-=4	d=d-4
=	e=5	e=e*5
/=	f/=3	f=f/3

Toán tử điều kiện

<điều kiện> ? <biểu thức 1> : <biểu thức 2>

Ví dụ:

```
int x = 10;
int y = 20;
int Z = (x<y) ? 30 : 40;
```

Kết quả: 30

Phép dịch trái: <<

$a \ll n$ Dịch tất cả các bit của a sang trái n lần, điền số 0 vào bên phải. ($a \ll n$ tương đương với $a * 2^n$).

Phép dịch phải và điền bit dấu: >>

$a \gg n$ Dịch tất cả các bit của a sang phải n lần, điền bit dấu vào bên trái.

Phép dịch phải và điền bit 0: >>>

$a \ggg n$ Dịch tất cả các bit của a sang phải n lần, điền 0 vào bên trái. $A \ggg n$ tương đương với $a * n/2$

Giá trị của đối số bên phải (ở trên là n) luôn là số nguyên (dương) do vậy đối số bên trái (ở trên là a) nếu là *byte* hoặc *short* thì phải đổi sang kiểu *int*. Kết quả của các phép chuyển dịch vì vậy sẽ luôn là *int* (hoặc là kiểu *long* nếu đối số thứ nhất là *long*).

Chuyển đổi kiểu:

- Ép kiểu

Qui tắc ép kiểu có dạng: (<type>) <exp>

Chuyển kết quả tính toán của biểu thức <exp> sang kiểu được ép là <type>.

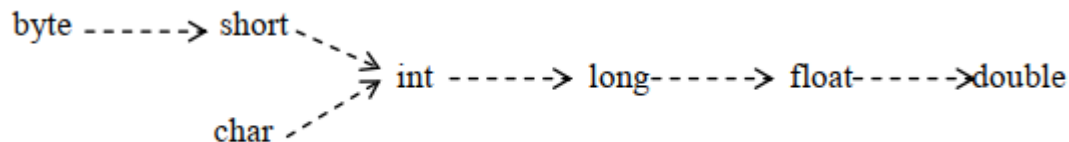
Ví dụ: float f = (float) 100.15D; // Chuyển số 100.15 dạng kiểu *double* sang *float*

Lưu ý:

- Không cho phép chuyển đổi giữa các kiểu nguyên thủy với kiểu tham chiếu, ví dụ kiểu *double* không thể ép sang các kiểu lớp như *HocSinh* được.
- Kiểu giá trị *boolean* (logic) không thể chuyển sang các kiểu dữ liệu số và ngược lại.

- **Mở rộng và thu hẹp kiểu**

Giá trị của kiểu hẹp hơn (chiếm số byte ít hơn) có thể được chuyển sang những kiểu rộng hơn (chiếm số byte nhiều hơn) mà không tổn thất thông tin. Cách chuyển kiểu đó được gọi là mở rộng kiểu



Hình Error! No text of specified style in document..2 Mở rộng kiểu

Ví dụ: `char c = "A";`

`int k = c; // mở rộng kiểu char sang kiểu int (mặc định)`

Chuyển đổi kiểu theo chiều ngược lại, từ kiểu rộng về kiểu hẹp hơn được gọi là *thu hẹp kiểu*. **Lưu ý** là *thu hẹp kiểu có thể dẫn tới mất thông tin*.

2.5.4 Các lệnh nhập/xuất dữ liệu qua thiết bị vào/ra chuẩn

❖ Nhập dữ liệu từ bàn phím

Java là một ngôn ngữ lập trình thuần hướng đối tượng, nên cách nhập dữ liệu từ bàn phím cũng phải sử dụng theo hướng đối tượng. Tức là ta phải khai báo một đối tượng, sau đó gọi hàm nhập, rồi gán giá trị cho biến.

Scanner là một lớp trong thư viện `Java.util` giúp nhập và lưu dữ liệu từ bàn phím. Ngoài `scanner`, còn có nhiều lớp khác nữa, nhưng đây là lớp được sử dụng nhiều nhất.

Các phương thức thường dùng trong lớp **Scanner**:

Tên phương thức	Tác dụng
<code>nextBoolean</code>	Nhập vào kiểu Boolean (true – false) từ bàn phím
<code>nextByte</code>	Nhập vào kiểu dữ liệu Byte
<code>nextShort</code>	Nhập vào kiểu Short (số nguyên từ -32768 đến 32767)
<code>nextInt</code>	Nhập vào kiểu số nguyên từ bàn phím
<code>nextFloat</code>	Nhập vào kiểu số thực
<code>nextDouble</code>	Nhập vào kiểu Double
<code>nextLine</code>	Nhập vào kiểu String
<code>nextLong</code>	Nhập vào số nguyên lớn

Cú pháp:

Scanner console = **new** Scanner(System.in);

console.nextInt(); // Gọi hàm nhập số nguyên. Các hàm khác cú pháp tương tự

❖ Xuất dữ liệu ra màn hình

Cú pháp

System.out.println("Nội dung cần in" + biểu thức);

System.out.print("Nội dung cần in" + biểu thức);

System.out.printf("Nội dung cần in" + biểu thức);

- Với *print*: Xuất kết quả ra màn hình nhưng con trỏ chuột không xuống dòng.
- Với *println*: Xuất kết quả ra màn hình đồng thời con trỏ chuột nhảy xuống dòng tiếp theo.
- Với *printf*: Xuất ra màn hình kết quả đồng thời có thể định dạng được kết quả đó nhờ vào các đối số thích hợp.

System.out.printf(local, format, arguments1, arguments2, ..., argumentsn);

Trong đó:

- *local*: Nếu khác null sẽ được tự động định dạng theo khu vực.
- *format*: Quy định chuẩn định dạng đầu ra cho các đối số
- Các *argument*: Đối số cần định dạng.

Các bộ định dạng có sẵn trong printf:

%c: Ký tự

%d: Số thập phân (số nguyên) (cơ số 10)

%e: Dấu phẩy động theo cấp số nhân

%f: Dấu phẩy động

%i: Số nguyên (cơ số 10)

%o: Số bát phân (cơ số 8)

%s: Chuỗi

%u: Số thập phân (số nguyên) không dấu

%x: Số trong hệ thập lục phân (cơ số 16)

%t: Định dạng ngày / giờ

%%: Dấu phần trăm

\\%: Dấu phần trăm

Ví dụ 1: Chương trình tính tổng hai số nguyên được nhập từ bàn phím và hiển thị tổng đó ra màn hình:

```
import java.util.Scanner;
```

```

public class SumTwoIntegers {
    public static void main(String[] args) {
        // Tạo đối tượng Scanner để nhận đầu vào từ bàn phím
        Scanner scanner = new Scanner(System.in);
        // Yêu cầu người dùng nhập số nguyên thứ nhất
        System.out.print("Nhập số nguyên thứ nhất: ");
        int number1 = scanner.nextInt();
        // Yêu cầu người dùng nhập số nguyên thứ hai
        System.out.print("Nhập số nguyên thứ hai: ");
        int number2 = scanner.nextInt();
        // Tính tổng hai số
        int sum = number1 + number2;
        // Hiển thị kết quả
        System.out.println("Tổng hai số nguyên là: " + sum);
        // Đóng Scanner để tránh rò rỉ tài nguyên
        scanner.close();
    }
}

```

2.5.6 Các câu lệnh điều khiển/có cấu trúc

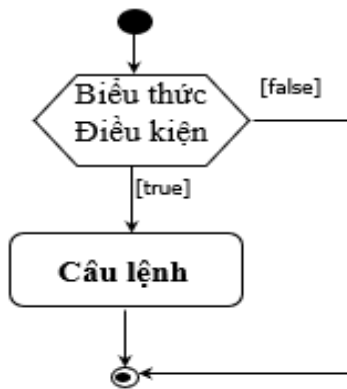
❖ Câu lệnh rẽ nhánh

Câu lệnh if đơn giản

if (<Biểu thức điều kiện>)

 <Câu lệnh>;

Nếu biểu thức điều kiện đúng thì thực hiện câu lệnh

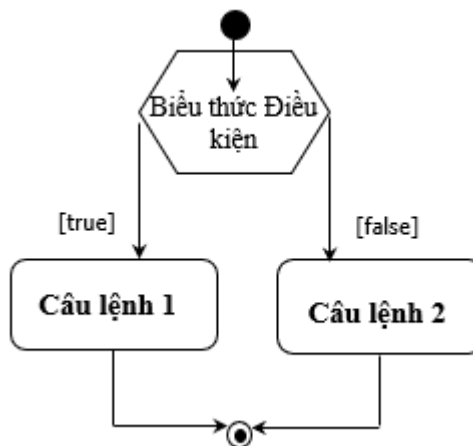


Hình 2Error! No text of specified style in document..**3 Câu lệnh if đơn giản**
Câu lệnh if ... else

```

if (<Biểu thức điều kiện>)
    <Câu lệnh 1>;
else
    <Câu lệnh 2>;
  
```

Nếu biểu thức điều kiện đúng thì thực hiện câu lệnh 1 nếu không thì thực hiện câu lệnh 2 (nếu có **else**)



Hình 2.4 Câu lệnh if ... else

Ví dụ 1:

```

public class Test {
    public static void main(String[] args) {
        int number = 13;
        if (number % 2 == 0) {
            System.out.println("Số " + number + " là số chẵn.");
        } else {
            System.out.println("Số " + number + " là số lẻ.");
        }
    }
}
  
```

Kết quả:

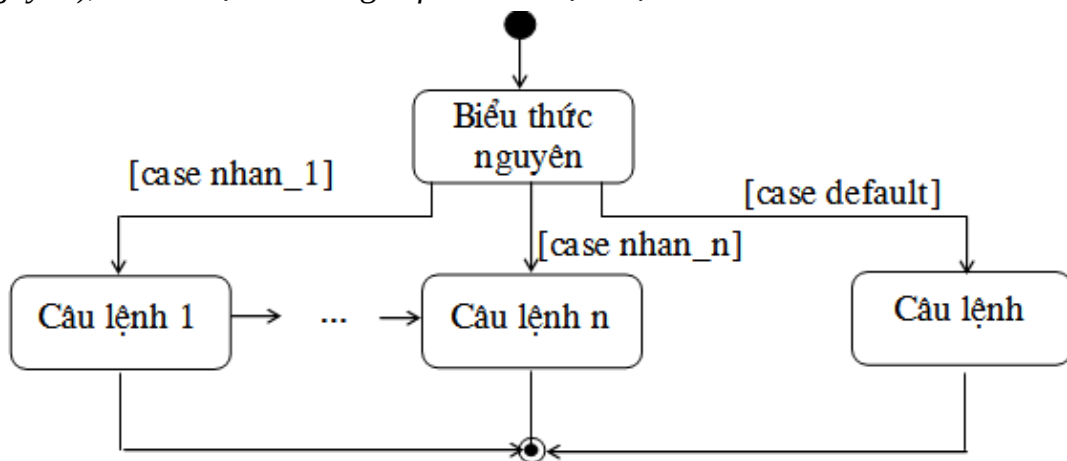
Số 13 là số lẻ.

Câu lệnh switch

```
switch (<Biểu thức nguyên>) {  
    case nhan_1: <Câu lệnh 1>;  
    case nhan_2: <Câu lệnh 2>;  
    ...  
    case nhan_n: <Câu lệnh n>;  
    [default: <Câu lệnh >];  
}
```

Câu lệnh *switch* sẽ kiểm tra giá trị của *Biểu thức nguyên* và so sánh với từng giá trị *nhan1*, *nhan2*, ..., lần lượt từ trên xuống dưới, mỗi giá trị cần so sánh gọi là một *case*. Khi một trường hợp đúng (true), khối lệnh ở trong *case* đó sẽ được thực thi.

Nếu tất cả các trường hợp đều sai (tức là các *nhan* ở *case* không bằng với *Biểu thức nguyên*), thì câu lệnh ở trong *default* sẽ được thực thi.



Hình Error! No text of specified style in document..5 Câu lệnh switch

Ví dụ 2:

```
public class SwitchExample {  
    public static void main(String[] args) {  
        int number = 20;  
        switch (number) {  
            case 10:  
                System.out.println("10");  
                break;  
            case 20:  
                System.out.println("20");  
                break;  
            case 30:  
                System.out.println("30");  
                break;  
        }  
    }  
}
```



```

        default:
            System.out.println("Not in 10, 20 or 30");
    }
}
}

```

Kết quả: 20

Lưu ý:

- Khởi *default* là không bắt buộc có ở cấu trúc *switch case* trong Java
- Trong một *switch* có thể có nhiều *case*
- Nếu không gặp lệnh *break* trong khối lệnh này, thì chương trình sẽ thực hiện tiếp các *case* bên dưới cho tới khi nó gặp lệnh *break* thì nó sẽ thoát ra khỏi *switch*, hoặc cho đến khi thực hiện hết câu lệnh.

Ví dụ 3: mệnh đề *switch-case* khi không sử dụng '*break*'

```

public class SwitchExample2 {
    public static void main(String[] args) {
        int number = 20;
        switch (number) {
            case 10:
                System.out.println("10");
            case 20:
                System.out.println("20");
            case 30:
                System.out.println("30");
            default:
                System.out.println("Not in 10, 20 or 30");
        }
    }
}

```

Kết quả:

20

30

Not in 10, 20 or 30

❖ Các câu lệnh lặp

Vòng lặp for

for (<Biểu thức bắt đầu>; <Điều kiện lặp>; <Biểu thức gia tăng>)
<Khối lệnh>

Trong đó:

- *Biểu thức bắt đầu*: được thực thi đầu tiên, và chỉ một lần. Bước này cho phép khai báo và khởi tạo bất kỳ biến điều khiển vòng lặp. Tất cả các biến được khai báo trong <Biểu thức bắt đầu> đều là cục bộ trong khối thân của chu trình **for**.
- *Điều kiện lặp*: Nếu nó là **true**, phần *Khối lệnh* được thực thi. Nếu nó là **false**, phần *Khối lệnh* không thực thi và luồng điều khiển nhảy tới lệnh tiếp theo sau vòng lặp **for**.
- *Biểu thức gia tăng*: Thay đổi (tăng hoặc giảm) giá trị của biến điều khiển sau mỗi lần lặp

Ví dụ 4:

```
public class Main {
    public static void main(String args[]) {
        for (int i = 0; i < 5; i++) {
            System.out.println("Vị trí thứ: " + i);
        }
    }
}
```

Kết quả:

Vị trí thứ 0

Vị trí thứ 1

Vị trí thứ 2

Vị trí thứ 3

Vị trí thứ 4

Lưu ý:

- Các thành phần của chu trình **for** là tùy chọn. Một trong <Biểu thức bắt đầu>, <Biểu thức gia tăng>, <Điều kiện kết thúc> có thể trống. Trường hợp <Điều kiện kết thúc> là trống thì điều kiện lặp của chu trình được xem là **true**.
- **for** (; ;) được sử dụng để xây dựng chu trình lặp vô điều kiện.

Vòng lặp foreach

```
for (<kiểu dữ liệu> <tên biến chạy> : <tên mảng>) {
    <Khối lệnh>
}
```

Vòng lặp *for-each* được sử dụng để duyệt qua từng phần tử trong một mảng hoặc một tập hợp. trong đó:

- <kiểu dữ liệu>: Kiểu dữ liệu của từng phần tử trong mảng hoặc tập hợp mà bạn muốn duyệt qua. Kiểu này phải trùng với kiểu của các phần tử trong mảng.

- *<tên biến chạy>*: Tên của biến sẽ đại diện cho từng phần tử trong mảng khi vòng lặp chạy qua. Biến này sẽ lần lượt nhận giá trị của từng phần tử trong mảng.
- *<tên mảng>*: Tên của mảng hoặc tập hợp chứa các phần tử mà bạn muốn duyệt qua.
- *<Khối lệnh>*: Là khối lệnh được thực thi cho mỗi phần tử trong mảng. Trong khối này, có thể sử dụng *<tên biến chạy>* để truy cập giá trị của phần tử hiện tại.

Ví dụ 5:

```
public class Main {
    public static void main(String args[]) {
        int[] mang = {4,3,2,1,0};
        for (int i : mang) {
            System.out.println("Giá trị thứ : " + i);
            i++;
        }
    }
}
```

Kết quả:

```
Giá trị thứ : 4
Giá trị thứ : 3
Giá trị thứ : 2
Giá trị thứ : 1
Giá trị thứ : 0
```

Lưu ý: Vòng lặp for-each chỉ cho phép duyệt qua các phần tử của mảng hoặc tập hợp; không thể thay đổi trực tiếp các phần tử trong mảng.

Vòng lặp while

```
while (<điều kiện>)
    <khối lệnh>
```

Trong đó:

- *điều kiện*: Biểu thức **boolean**, nó trả về giá trị **true** hoặc **false**. Vòng lặp sẽ tiếp tục cho đến khi nào giá trị **true** được trả về
- *khối lệnh*: Các câu lệnh được thực hiện nếu *điều kiện* nhận giá trị True

Ví dụ 6:

```
public class Main {
    public static void main(String args[]) {
        int i=0;
        while(i<5){
            System.out.println("Vị trí thứ "+i);
            i++;
        }
    }
}
```

```
}
```

Vòng lặp **while** kiểm tra điều kiện nếu $i < 5$ thì in ra vị trí i . Mỗi lần lặp, chúng ta tăng i thêm 1 đơn vị, do đó, sau 5 lần lặp, i có giá trị là 5 và vòng lặp sẽ dừng lại.

Kết quả:

Vị trí thứ 0

Vị trí thứ 1

Vị trí thứ 2

Vị trí thứ 3

Vị trí thứ 4

Vòng lặp do...while

Vòng lặp do ... while là tương tự như Vòng lặp while, ngoại trừ rằng phần thân của vòng lặp do...while được bảo đảm thực thi ít nhất một lần. Nói cách khác, vòng lặp do ... while thực hiện phần thân vòng lặp trước khi kiểm tra điều kiện.

```
do{
```

```
<khối lệnh>
```

```
}while (<điều kiện>)
```

Trong đó:

- *điều kiện*: Biểu thức **boolean**, nó trả về giá trị **true** hoặc **false**. Vòng lặp sẽ tiếp tục cho đến khi nào giá trị **true** được trả về
- *khối lệnh*: Các câu lệnh được thực hiện 1 lần sau đó kiểm tra điều kiện, nếu đúng thì thực hiện tiếp và ngược lại.

Ví dụ 7:

```
public class Main {  
    public static void main(String args[]) {  
        int i = 0;  
        do {  
            System.out.println("Vị trí thứ : " + i);  
            i++;  
        } while (i < 5);  
    }  
}
```

Kết quả:

Vị trí thứ 0

Vị trí thứ 1

Vị trí thứ 2

Vị trí thứ 3

Vị trí thứ 4

❖ Các lệnh chuyển vị khác

Câu lệnh break:

Câu lệnh **break** được sử dụng trong các khối được gắn nhãn, trong các chu trình lặp (**for**, **while**, **do-while**) và câu lệnh **switch** để chuyển điều khiển thực hiện chương trình ra khỏi khối trong cùng chứa nó.

- Trong gắn nhãn: phần còn lại của khối bị bỏ qua
- Trong chu trình lặp: khi gặp lệnh break thì phần còn lại của thân chu trình được bỏ qua và kết thúc chu trình đó,
- Trong lệnh **switch**: phần còn lại của lệnh **switch** bị bỏ qua và tiếp tục thực hiện lệnh đứng sau lệnh **switch** đó.

Ví dụ 8:

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                break;  
            }  
            System.out.println(i);  
        }  
    }  
}
```

Kết quả:

1
2
3
4

Ví dụ 9: sử dụng break với vòng lặp bên trong vòng lặp for khác:

```
public class BreakExample2 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {  
                    break;  
                }  
            }  
        }  
    }  
}
```

```

        System.out.println(i + " " + j);
    }
}
}

```

Kết quả:

```

1 1
1 2
1 3
2 1
3 1
3 2
3 3

```

Câu lệnh continue

Câu lệnh **continue** được sử dụng trong các chu trình lặp **for**, **while**, **do-while** để dừng sự thực hiện của lần lặp hiện thời và bắt đầu lặp lại lần tiếp theo nếu điều kiện lặp còn thoả mãn (*true*).

Ví dụ 10: sử dụng **continue** trong java với vòng lặp **for**:

```

public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                continue;
            }
            // Khi i == 5 thì không in i = 5 ra màn hình
            System.out.println(i);
        }
    }
}

```

Kết quả:

```

1
2
3
4
6

```

7
8
9
10

Ví dụ 11: sử dụng **continue** với vòng lặp bên trong vòng lặp **for** khác:

```
public class ContinueExample2 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {  
                    continue;  
                }  
                // Không in trường hợp i=2 và j=2 ra màn hình  
                System.out.println(i + " " + j);  
            }  
        }  
    }  
}
```

Kết quả:

1 1
1 2
1 3
2 1
2 3
3 1
3 2
3 3

Câu lệnh return

Câu lệnh **return** được sử dụng để kết thúc thực hiện của hàm hiện thời và chuyển điều khiển chương trình về lại sau lời gọi hàm đó.

Bảng Error! No text of specified style in document..4 **Dạng câu lệnh return**

Dạng câu lệnh return	Hàm khai báo void	Hàm có kiểu trả lại khác void
return	Tùy chọn	Không cho phép
return <Biểu thức>	Không cho phép	Bắt buộc

Tổng kết bài học

Bài học này cung cấp một cái nhìn tổng quan và chi tiết về ngôn ngữ lập trình Java:

- Lịch sử phát triển Java: Java đã phát triển qua nhiều giai đoạn từ khi được tạo ra, không ngừng cải tiến và trở thành một ngôn ngữ linh hoạt, đa nền tảng, được ưa chuộng trong nhiều lĩnh vực, đặc biệt là trong các hệ thống phân tán và ứng dụng doanh nghiệp.
- Cấu trúc chương trình Java: Cấu trúc của một chương trình Java bao gồm các lớp và đối tượng là những thành phần cơ bản. Mỗi chương trình Java sẽ bắt đầu từ một lớp chính (main class) chứa phương thức main(), nơi chương trình bắt đầu thực thi.
- Biên dịch và thông dịch Java: Java sử dụng mô hình biên dịch kết hợp với thông dịch, qua đó mã nguồn Java được biên dịch thành bytecode và thực thi bởi JVM. Điều này giúp chương trình Java có thể chạy trên bất kỳ nền tảng nào mà không cần chỉnh sửa.
- Các dạng ứng dụng Java: Java có thể được sử dụng để phát triển nhiều loại ứng dụng khác nhau, từ ứng dụng độc lập, ứng dụng web, cho đến các ứng dụng phân tán, doanh nghiệp và các hệ thống tích hợp cơ sở dữ liệu.

Bài học cũng giới thiệu các thành phần cơ bản trong lập trình Java, bao gồm:

- Các phần tử cơ sở của Java: Giới thiệu những khái niệm nền tảng của ngôn ngữ Java.
- Các kiểu dữ liệu trong Java: Trình bày các kiểu dữ liệu cơ bản như kiểu nguyên thủy (int, float, char...) và kiểu tham chiếu (String, Array...).
- Biến, hằng, biểu thức và các phép toán cơ bản: Bao gồm cách khai báo và sử dụng biến, hằng, cùng với các phép toán cơ bản như toán tử số học, so sánh và logic.
- Câu lệnh nhập xuất dữ liệu cơ bản: Hướng dẫn cách nhập dữ liệu từ bàn phím và xuất dữ liệu ra màn hình.
- Các cấu trúc điều khiển chương trình: Giới thiệu các cấu trúc điều khiển như câu lệnh rẽ nhánh (if, switch), các câu lệnh lặp (for, while, do-while) và các lệnh chuyển vị (như break, continue).

Nội dung thực hành

Bài tập 1: Viết chương trình nhập vào 3 số nguyên từ bàn phím, chương trình sẽ in ra số nguyên có giá trị lớn nhất

Bài tập 2: Viết chương trình nhập và 1 số nguyên từ bàn phím, chương trình sẽ in ra thông tin số vừa nhập là số 'âm', 'dương' hoặc là số 'không'

Bài tập 3: Viết chương trình cho phép nhập vào 3 số thực tương ứng với độ dài của một tam giác và kiểm tra 3 số này có phải là 3 cạnh của một tam giác hay không (nếu nó thỏa mãn điều kiện tổng của hai cạnh bất kỳ đều lớn hơn cạnh thứ 3) .

Bài tập 4: Với n nhập từ bàn phím hay viết các chương trình tính tổng các dãy sau:

1. Tính tổng dãy

$$S=1+2-3+\dots+(-1)^{n+1}.n$$

2. Tính tổng dãy:

$$S=1!+2!+3!+\dots+n!$$

3. Nếu n lẻ: tính tổng các số lẻ $<n$, nếu n chẵn: tính tổng các số chẵn $<n$.

4.
$$S = \frac{1}{2} + \frac{2}{3} + \dots + \frac{n-1}{n}$$

Bài tập 5: Sử dụng cấu trúc try, và 2 khối catch xây dựng chương trình nhập vào 2 số nguyên a, b và in ra kết quả a/b. Yêu cầu bắt các ngoại lệ NumberFormatException khi nhập a và b, ArithmeticException khi thực hiện a/b.

Bài tập 6: Dùng mệnh đề throw để tạo ra ngoại lệ khi nhập tuổi cho nhân viên (không âm và không lớn hơn 62 tuổi)

Bài tập 7: Xây dựng hàm nhapTuoi() và dùng mệnh đề Throws để chuyển ngoại lệ ra ngoài hàm và thực hiện bắt và xử lý các ngoại lệ khi gọi hàm nhapTuoi().